QCon
全球软件开发大会

InfoQ 技术大会

从Rollout到权重同步：Mooncake如何支撑高性能强化学习系统

赵鑫鹏
阿里云基础软件部技术专家 · Mooncake 社区 Code Owner

QCon 上海 2026



QCon
全球软件开发大会

InfoQ 技术大会

目录

01 Mooncake 简介

Store 与 TE / RL 全景 / Mooncake 场景 / KVCache

02 Rollout 数据传输

异步读写 / 碎片聚合 / 按需消费

03 RL 权重管理

容量回迁 / Tensor 读写 / 权重更新

04 未来展望

Agentic RL / RL 解耦 / 共享容量 / 可靠传输

01

Mooncake 简介

从 KVCache 到 RL 的存储与传输

QCon 全球软件开发大会
InfoQ 技术大会

Mooncake：Store 与 Transfer Engine

从 KVCache 起步，把对象管理与大块数据搬运分开

KVCache 的需求　　长上下文 / 多轮复用 / Prefill 与 Decode 分离

Mooncake Store

按 key 管理对象位置、副本与生命周期；利用 DRAM、SSD 等容量层

↓

Transfer Engine

已知内存区间之间搬运；registered memory / batching / device 与 network transport

GPU HBM

当前活跃计算

Host DRAM

共享容量与快速回迁

SSD / 远端容量

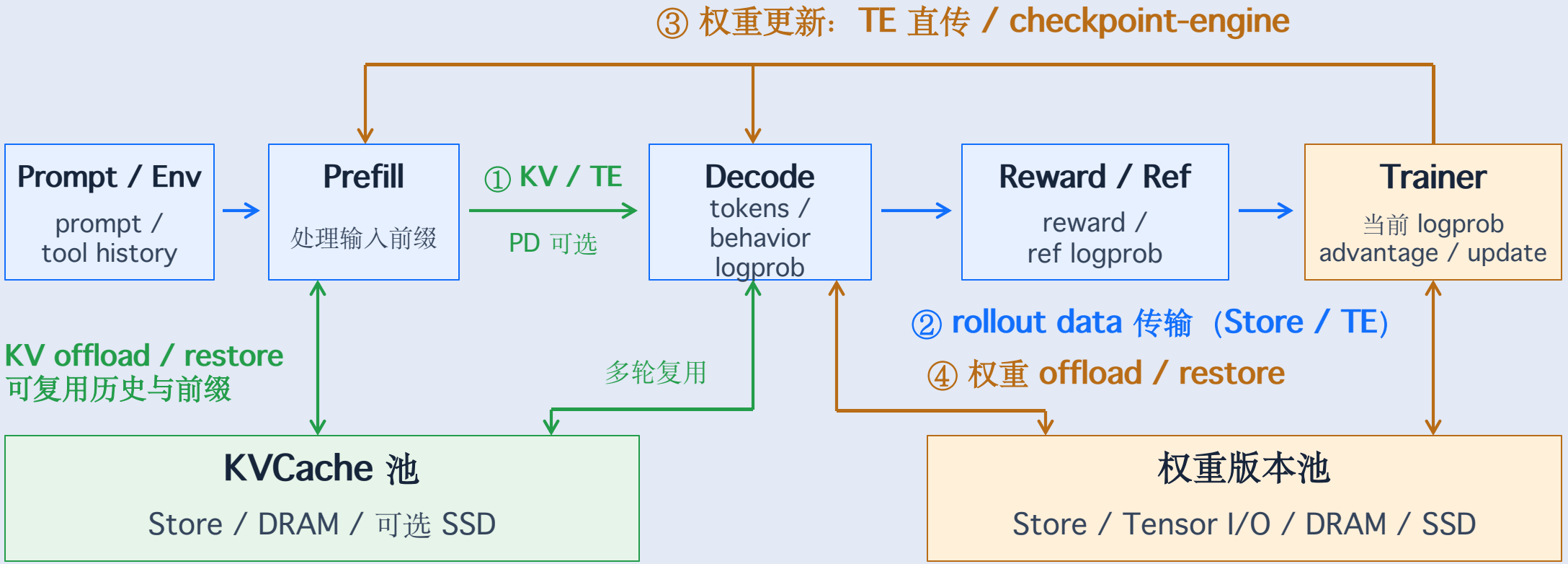
更深容量与跨节点访问

需要按对象找数据时使用 Store；源与目的地址明确时，也可以直接使用 Transfer Engine。

QCon 全球软件开发大会
InfoQ 技术大会

RL 数据流全景

以 GRPO 为例，覆盖推理内 KV 流转、训练输入与权重管理



HBM 留给活跃计算，Store 管对象与容量，TE 搬运内存区间

可选场景总览，并非全部同时启用。PRIME-RL 的 P→D 用 NIXL；图中 Mooncake PD 以 slime / SGLang 为例。

QCon 全球软件开发大会
InfoQ 技术大会

Mooncake 与 RL 场景

Rollout 本身需要推理，RL 同时需要样本传输、权重驻留与新策略发布

场景	主要成本	Mooncake 接入	直接收益
Rollout 推理	KV 工作集增长 前缀重算 / P-D 迁移	Store：PRIME-RL KV offload TE：slime / SGLang PD	扩展 KV 容量与复用 P/D 分别配置资源
Rollout data 传输	异构字段 / 碎片 序列化 / 重建 / 重复读	结构化对象 / Bulk I/O Store / TE	降低传输与暂存成本 trainer 更早拿到输入
权重驻留 / Offload	HBM / DRAM 紧张 不活跃权重占用容量	Tensor I/O DRAM / SSD 分层	释放实际可用空间 权衡恢复代价
新权重发布	周期更新 / 多接收端 fan-out / 更新停顿	registered buffer + TE checkpoint-engine P2P	减少重复发送与等待 更早使用新策略

RL 框架保留算法准入、调度、背压与版本策略

KVCache、rollout data 与权重的消费者和生命周期不同；数据面优化不消除生成长尾或环境耗时。

什么时候需要在 RL 中管理 KVCache

不是 RL 默认项，先看工作集、重算成本与部署方式

多轮 Agent

工具与环境反馈持续追加历史
下一轮复用上轮 Decode KV

高并发 / 长响应

峰值并发 token 放大 KV 工作集
频繁淘汰后仍会再次访问

P/D 分离

Prefill 与 Decode 独立部署
必须新增 KV 传输路径

使用条件

HBM 足够
本地 cache

→

反复淘汰 + 后续复用
Store offload / restore

→

P/D 分开
Mooncake TE 传输 KV

公开实现

KV 复用 / Offload
PRIME-RL · Mooncake Store

P/D 传输
slime / SGLang · Mooncake TE

关键指标：KV 淘汰率 · 重算时间 · 命中率 · 回迁成本

PRIME-RL：Mooncake Store 负责 KV 复用，P→D 使用 NIXLConnector / UCX；slime / SGLang：Mooncake TE 负责 P/D 传输。

02

Rollout data 传输

异步读写、碎片聚合与按需消费

QCon 全球软件开发大会
InfoQ 技术大会

Rollout data：从生成轨迹到训练样本

字段由算法和阶段决定，通常同时包含大 Tensor 与小元数据

阶段	典型字段	主要形态	常见生产者
输入与生成	prompt / input_ids response tokens / mask	变长 Tensor	Rollout engine
策略统计	behavior / old logprob 可选 values	Tensor [B, L]	Rollout / policy
评估与环境	reward / score tool result / error	Tensor + 小对象	RM / Env / Tools
训练准备	ref logprob / advantage / return loss mask	Tensor [B, L]	Reference / Trainer
身份与版本	sample / group / trajectory id policy version / metadata	标量 + metadata	Orchestrator

大头：Tensor Payload

小头：Schema / ID / 状态

字段跨阶段补齐，不代表所有算法都同时消费全部字段

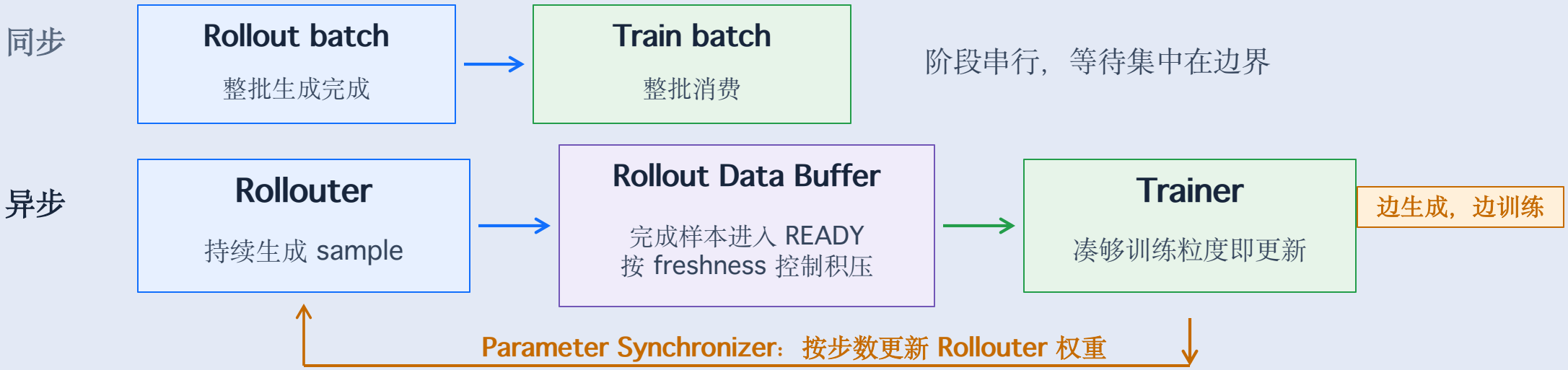
ROLL、PRIME-RL / verifiers 等公开实现用于核对字段；advantage 与 return 通常在训练准备阶段生成。

QCon 全球软件开发大会

InfoQ 技术大会

异步 RL 对 Rollout data 的新要求

Rollouter 持续生产，Trainer 持续消费，生成与训练在时间线上并行



异步可见

完成的样本或字段
先进入可读状态

版本新鲜度

policy version / age
决定训练准入

消费后回收

确认后续不再使用
框架主动 DELETE

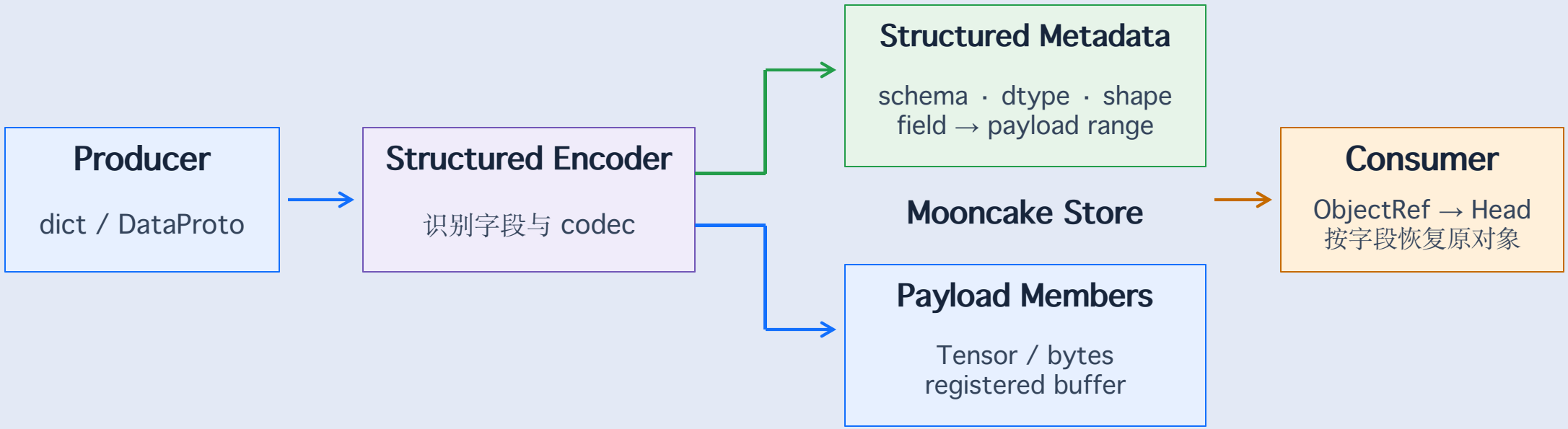
共同模式：持续生产、缓冲解耦、持续消费

流程抽象参考 VERL fully_async_policy；AReaL 同样将 rollout generation 与 model training 并行执行。

QCon 全球软件开发大会
InfoQ 技术大会

结构化传输：Metadata 与 Payload 分离

调度面只传轻量引用，数据面用 Mooncake 搬运实际 Payload



控制面：ObjectRef　数据面：Payload　框架：依赖与版本　Mooncake：存储与搬运

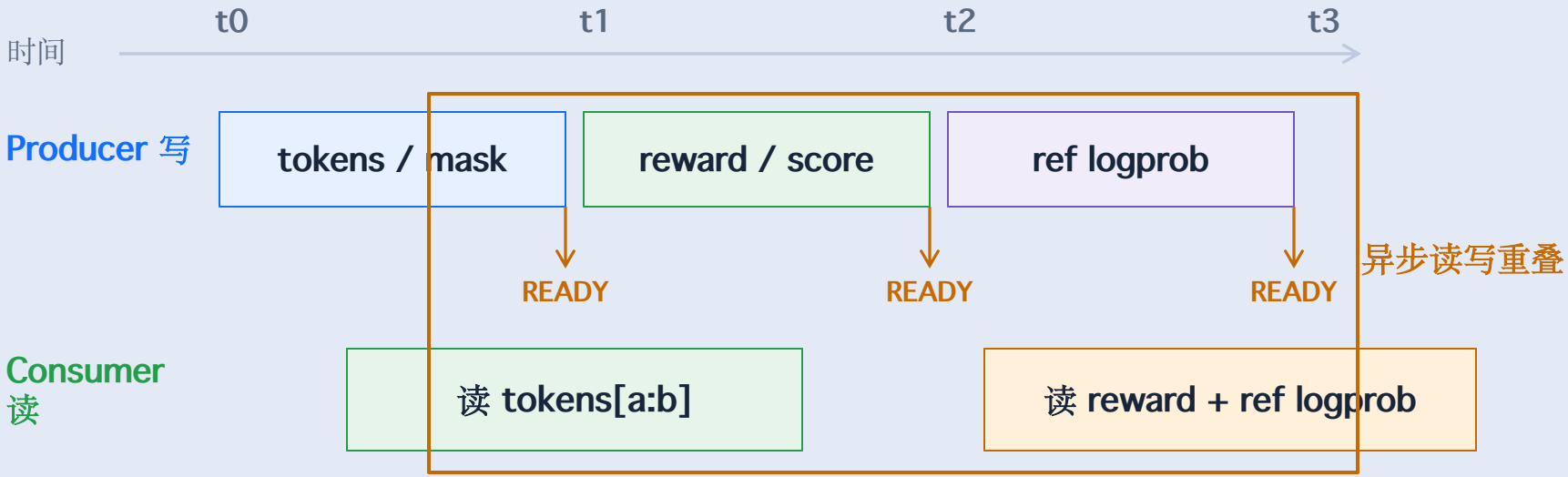
先解决表示与寻址，再处理小 I/O 固定开销和字段就绪

Miles 当前验证 completed-dict 整批路径；字段级发布属于上层协议设计。

QCon 全球软件开发大会
InfoQ 技术大会

分阶段发布与按需读取

Producer 写后续字段时，Consumer 可以读取已经 READY 的字段或 range



按需读取

字段

response + reward

字段内 range

response[a:b]

多字段 ranges

tokens[a:b]
mask[c:d]

读取计划只覆盖当前依赖的数据，写入后续字段不必阻塞已就绪字段的消费

框架维护字段状态与训练准入；Mooncake 按字段和 ranges 搬运已经发布的 Payload。

QCon 全球软件开发大会
InfoQ 技术大会

Rollout data 碎片化放大传输固定开销

变长字段、独立 allocation 与错峰就绪，让小 I/O、注册和调度次数增加

样本行	Response	Mask	Logprob	Reward / Metadata
sample 0	Tensor [L0]	Tensor [L0]	Tensor [L0]	scalar + id
sample 1	Tensor [L1]	Tensor [L1]	Tensor [L1]	tool result + id
sample 2	Tensor [L2]	Tensor [L2]	可选 / 后到	scalar + version

逻辑 Batch



变长序列

可选字段

分组 / 过滤 / 切分

不同就绪时间

问题：小 I/O 数量增加 · 内存注册频繁 · 序列化与重建开销

逐块传输放大固定开销；整批 pickle / concat 又会增加临时副本。

QCon 全球软件开发大会
InfoQ 技术大会

碎片聚合：Buffer Pool 与 Bulk I/O

复用注册内存、批量提交；可用路径直接读写 caller buffer

逻辑字段

response　mask　logprob

reward　metadata　ref logprob

聚合器 →

注册 Buffer Pool / 物理 Chunks

range 1	range 2	range 3	range 4	range 5

Bulk PUT / GET · Payload 直接读写注册 Buffer

优化点	为什么有效	边界
聚合小 payload	减少请求与提交次数，避免大量小块逐个 register	字段仍由 offset + length 定位
注册 Buffer Pool	复用已注册内存，减少 alloc / free 与 register / unregister	容量和租约需要上限
from / into 零拷贝路径	Payload 直接读写 caller buffer，减少中间 bytes / staging	要求连续、已注册且路径支持
Bulk 并行 I/O	多个 chunk 并发提交，提高有效带宽	并行度受设备与队列限制

Ragged 数据仍可能需要必要的聚合复制；这里的零拷贝指省掉中间 bytes 和 staging。

Rollout data 的生命周期与主动回收

写入与发布

PUT 完成
引用可读

→

读取与计算

按需 GET
计算后释放本地结果

→

框架确认

当前消费已完成
后续不会再消费

→

主动 DELETE

清理 payload
与 manifest

判断位置	正确条件
可发布	payload 与必要描述写入完成，再发布引用
可使用	读取完成，目的 buffer 可用，再交给计算
可删除	全部当前消费者结束，且后续训练 / 重读 / 重试不再需要

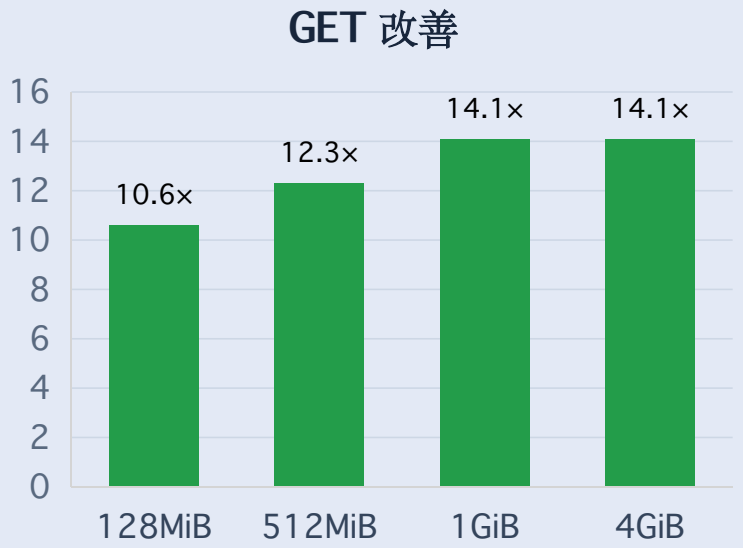
收益：用完即释放，避免已消费对象持续占用 DRAM

消费确认由框架负责；GET 返回或本地 buffer 释放，都不能单独作为删除 Store 对象的条件。

Rollout data 传输性能与训练等待

Miles completed-dict，对照 Ray backend 的传统序列化路径；计时为 payload 传输 + 重建

Payload	传统序列化 GET	MC GET	改善
128 MiB	501 ms	47 ms	10.6×
512 MiB	2,134 ms	173 ms	12.3×
1 GiB	5,035 ms	356 ms	14.1×
4 GiB	20,034 ms	1,426 ms	14.1×



PUT：1.2–1.6×

组件改善 ≠ RL 吞吐同倍提升

生成 / 计算
可能覆盖传输

暴露 GET 等待
只有这段缩短才直接省本轮时间

8 条来源样本规范化后受控放大，保留逐行碎片；非生产 batch 分布。计时不含 reference / scheduler，未做机制消融。

QCon 全球软件开发大会
InfoQ 技术大会

03

RL 权重管理

Offload / Restore 与权重更新

QCon 全球软件开发大会
InfoQ 技术大会

两种典型部署中的权重驻留与 Offload

同步共卡按阶段切换 HBM；异步分卡分别管理两组 GPU 的活跃与非活跃权重

同步共卡

同一组 GPU · 阶段串行

Rollout 阶段

推理权重 + KVCache 驻留 HBM
训练参数 / optimizer state 下沉 DRAM

Engine Switching

Update 阶段

训练参数 / optimizer state 恢复 HBM
推理引擎释放 GPU 资源

Offload / Restore 是阶段切换的一部分

异步分卡

两组 GPU · 并行执行

Rollout GPU Pool

Active policy N 常驻
持续生成样本

新版本 ←

Trainer GPU Pool

训练 N / N+1
持续更新参数

活跃权重　　留在各自 HBM

旧版本 / 冷状态　　按需下沉 DRAM / SSD

Offload 管容量；权重同步负责版本发布

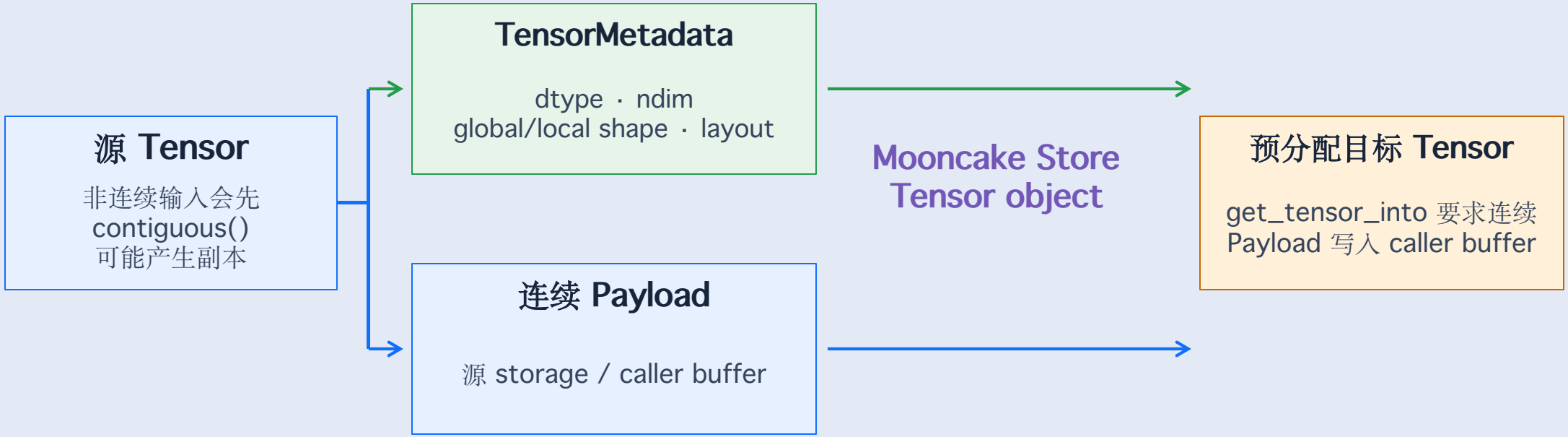
同步共卡看切换成本；异步分卡看驻留预算与版本更新

典型模式来自公开 RL 系统：task-collocated 是时间复用，task-separated 是空间拆分。

QCon 全球软件开发大会
InfoQ 技术大会

Tensor 零拷贝 I/O：适用路径与条件

普通 Store Tensor object：Metadata + 连续 Payload；是否零拷贝取决于连续性、注册与后端路径



普通 put / get：条件满足时减少 staging

direct TE：地址 + byte ranges

canonical snapshot：manifest / fragment

QCon 全球软件开发大会
InfoQ 技术大会

同节点数据路径：CUDA IPC 与 pinned memory

优先选择当前节点的 segment，再根据源端和目标端位置选择数据路径

local_first / 调用方指定本地 preferred_segment(s)

单 MEMORY replica 的本地优先是 best effort；需要稳定 host identity

CUDA IPC

跨进程访问同一 GPU allocation
GPU↔GPU 可免 Host 中转
GPU↔DRAM 仍需 DMA

Dummy 调接口 → Real 提供 segment

Pinned memory

比 pageable memory 更适合异步 DMA
H2D / D2H 可打满 PCIe 有效带宽

GPU ⟷ DRAM

可打满 PCIe 有效带宽

CUDA IPC 解决跨进程访问；pinned memory 提升必要 H2D / D2H DMA 的吞吐。

QCon 全球软件开发大会
InfoQ 技术大会

Store 的 HCA / NUMA 亲和放置与读取

NUMA node 指单机内部拓扑域；Put 选择内存 range，Get 选择已有副本

Store：placement / replica selection　　Transfer Engine：实际传输与 RDMA rail 选择

PUT / UPSERT：选择目标 NUMA range

请求源 Buffer

GPU / DRAM
HCA: mlx5_0

HCA 提示

同一 logical segment

NUMA node
0 · mlx5_0

NUMA node
1 · mlx5_1

匹配 HCA-local range，再按既有策略回退

GET / BATCH GET：读取副本选择

已有完整副本

Replica A · mlx5_0

Replica B · mlx5_1

当前目的 Buffer

GPU / DRAM
HCA: mlx5_1

只选择已有副本，不触发复制或迁移

NUMA 编号不跨主机比较；跨主机只用兼容 HCA 名称表达对应关系

适用于 rollout payload、Tensor object 与权重对象等 Store Put / Get 路径。

QCon 全球软件开发大会
InfoQ 技术大会

SSD 扩展权重容量

HBM 与 DRAM 都紧张时，按权重版本组下沉并恢复

驻留层级

HBM
ACTIVE

↕

DRAM
HOT / MIXED

↕

SSD
COLD

权重版本组

Immutable Revision
manifest + payload group

Evict
确认 cold copy 可读
再移除内存副本

Rehydrate
按 group 恢复
状态可为 HOT / COLD / MIXED

I/O 路径

Store SSD
可配置 io_uring
配合 bucket / batch I/O
提高磁盘利用率

C++ Transfer Engine GDS
专用 NVMe ↔ GPU 直传路径
依赖部署与设备栈

并行 I/O 目标：打满磁盘带宽

权重版本按 payload group 管理；Store 与 TE 分别承担容量和传输路径。

QCon 全球软件开发大会
InfoQ 技术大会

权重管理

不可变 revision、manifest 与 payload group 共同定义一个可读版本

Weight Metadata Store

精确 revision
availability / residency / lease

StoredWeightManifest

Tensor descriptor
fragment → object ranges

Store Object Metadata

物理副本位置
replica status

操作	顺序与约束	设计要求 / 可观察结果
Import	Begin → payloads → manifest last → commit	READY 后才可发现
Load	exact revision → lease → manifest / ranges → release	读取完整、精确版本
Evict / Rehydrate	先有可读 cold copy；按 payload group 转移	HOT / COLD / MIXED
Delete	lease 与引用结束后删除整个 group	不回收仍在使用的版本

Mooncake 目标：可读与可回收

RL 框架管 active policy 与请求切换

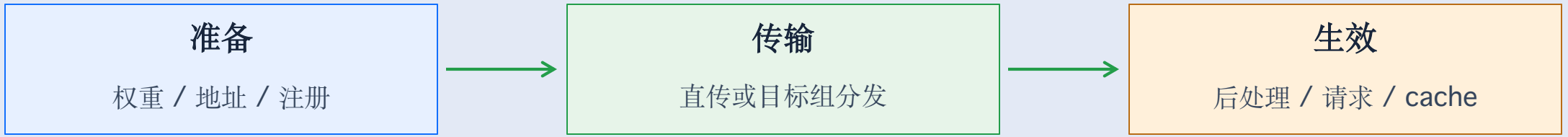
Mooncake 管版本完整性、驻留与回收；RL 框架负责 active policy 与请求切换。

QCon 全球软件开发大会
InfoQ 技术大会

权重同步的两种路径

从已准备好的权重开始，按发送端与接收端规模组织分发

适用条件	主要矛盾	建议数据路径
训练节点 ≥ 推理节点	源与目的映射明确 减少中间转存与协调	Mooncake TE 一对一直传
训练节点 < 推理节点	源端出口与重复发送 更多目标 / 局部加入	checkpoint-engine 组织分发 Mooncake 承担指定组 P2P



目标：减少源端重复发送，并缩短推理更新停顿

节点数是场景线索，不是唯一选型条件；checkpoint-engine 全员默认 Broadcast，指定 ranks 的 P2P 才调用 Mooncake。

QCon 全球软件开发大会
InfoQ 技术大会

TE 直传：RL 框架如何编排

控制面建立 endpoint 与映射，数据面在注册 GPU buffer 之间传输

控制面

1 接收端准备
GPU staging
注册地址与容量

→

2 上报 endpoint
worker → framework

→

3 建立映射
trainer rank → receiver(s)

→

4 下发计划
地址 / bucket / NIC

数据面

Lock + Pack
收集并转换参数
写入注册 buffer

→

TE Write
一对一或 fan-out
跨节点 RDMA

→

Tensor View
metadata 重建视图
load_weights

→

ACK + Unlock
全部接收完成
再复用 staging

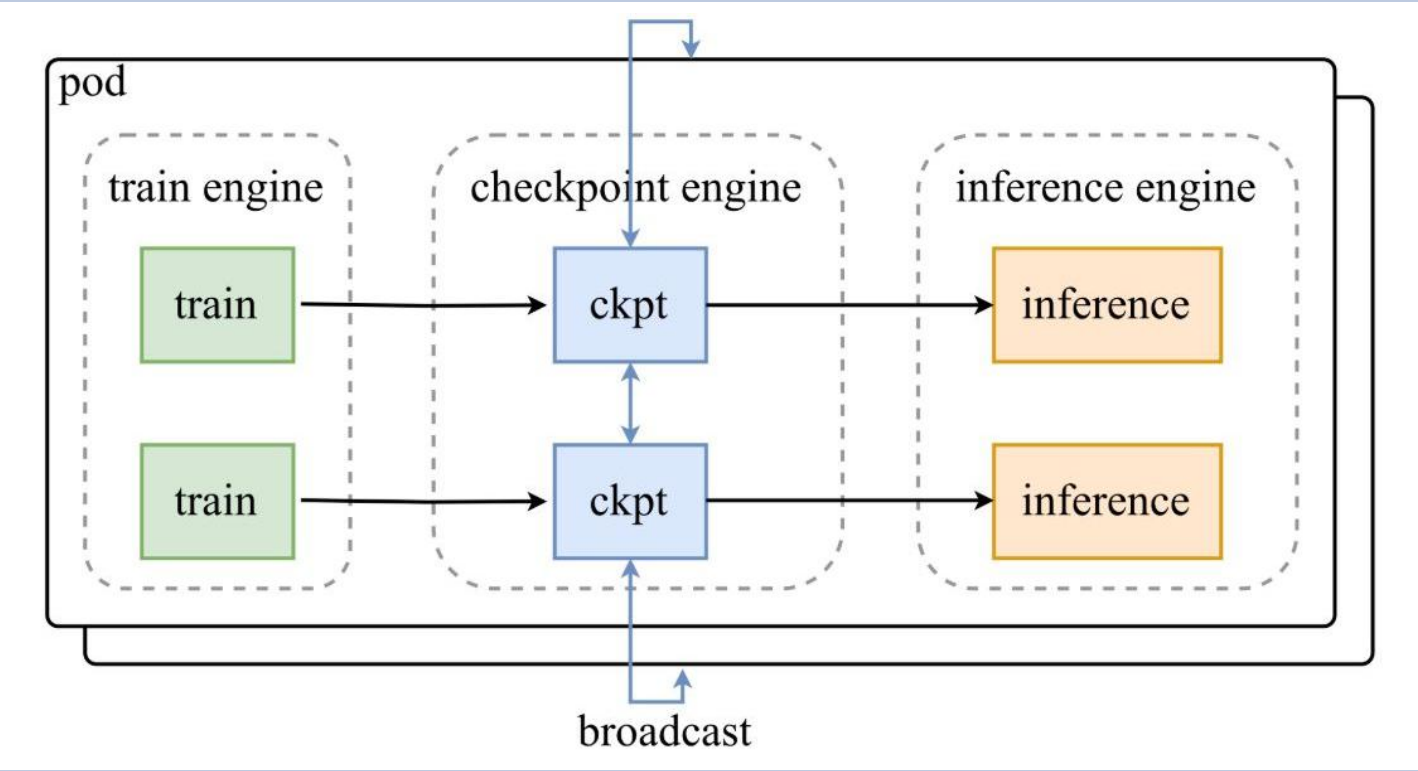
接收端重启：刷新 endpoint，再重试一次

现有 RL 框架代码还处理锁、分桶、endpoint 失效与重试；Mooncake 负责地址区间的高效传输。

QCon 全球软件开发大会
InfoQ 技术大会

checkpoint-engine：权重分发架构

全量更新走 Broadcast；指定 ranks 的 P2P 用于新实例加入和故障恢复



两条更新路径

Broadcast

全量周期更新
默认路径

指定 ranks 的 P2P

已有 CPU 权重副本
→ 新实例 / 重启实例

Mooncake TE：P2P 数据传输

checkpoint-engine 负责 metadata、bucket、目标 ranks 与引擎加载

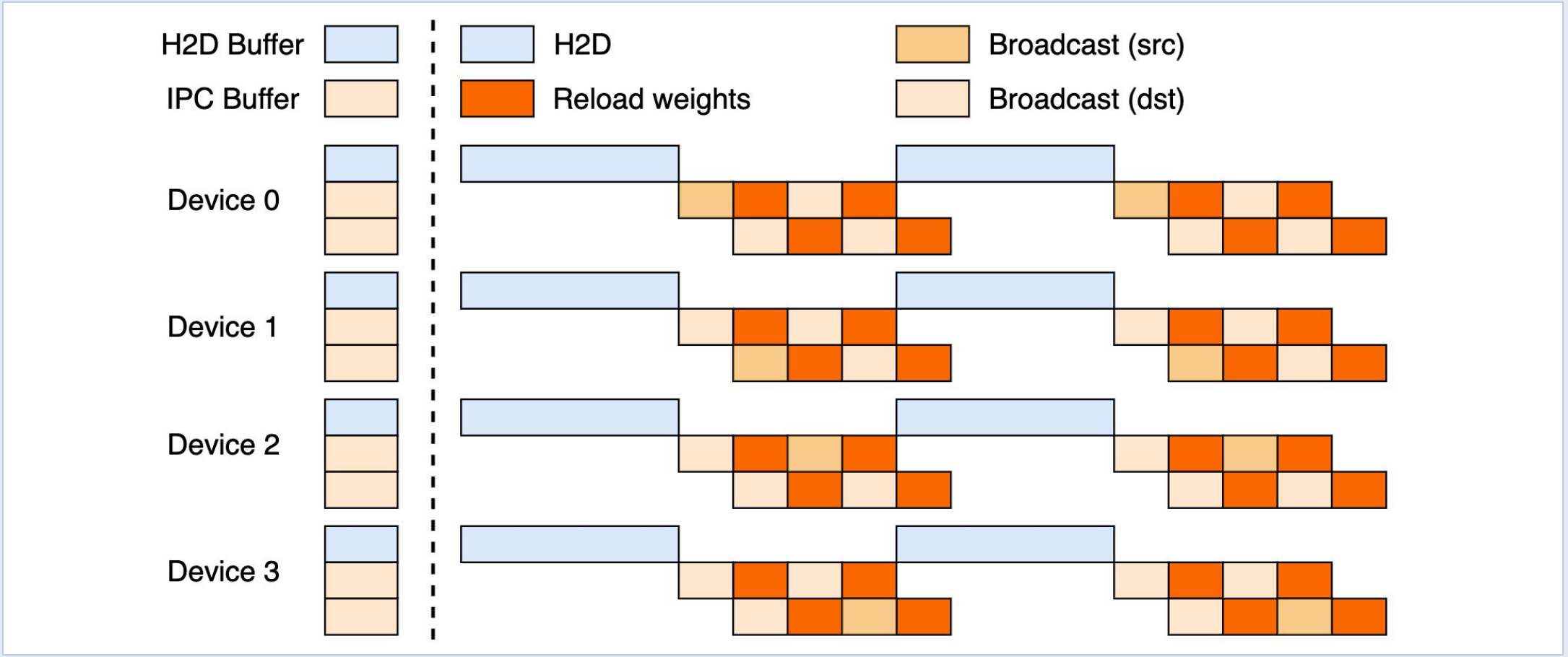
Mooncake 只进入 P2P 数据面；全员同步仍走 Broadcast

图片来源：MoonshotAI/checkpoint-engine README，保留社区原图。

QCon 全球软件开发大会
InfoQ 技术大会

checkpoint-engine：H2D、Broadcast 与 Reload 流水

按 bucket 在多设备上组织 H2D、广播与权重加载



图中是全量 Broadcast 流水；指定 ranks 的局部 P2P 由 Mooncake TE 传输跨节点 bucket。

QCon 全球软件开发大会
InfoQ 技术大会

权重传输完成与策略生效

目标是按约定运行新策略，不只是把字节写进目标地址



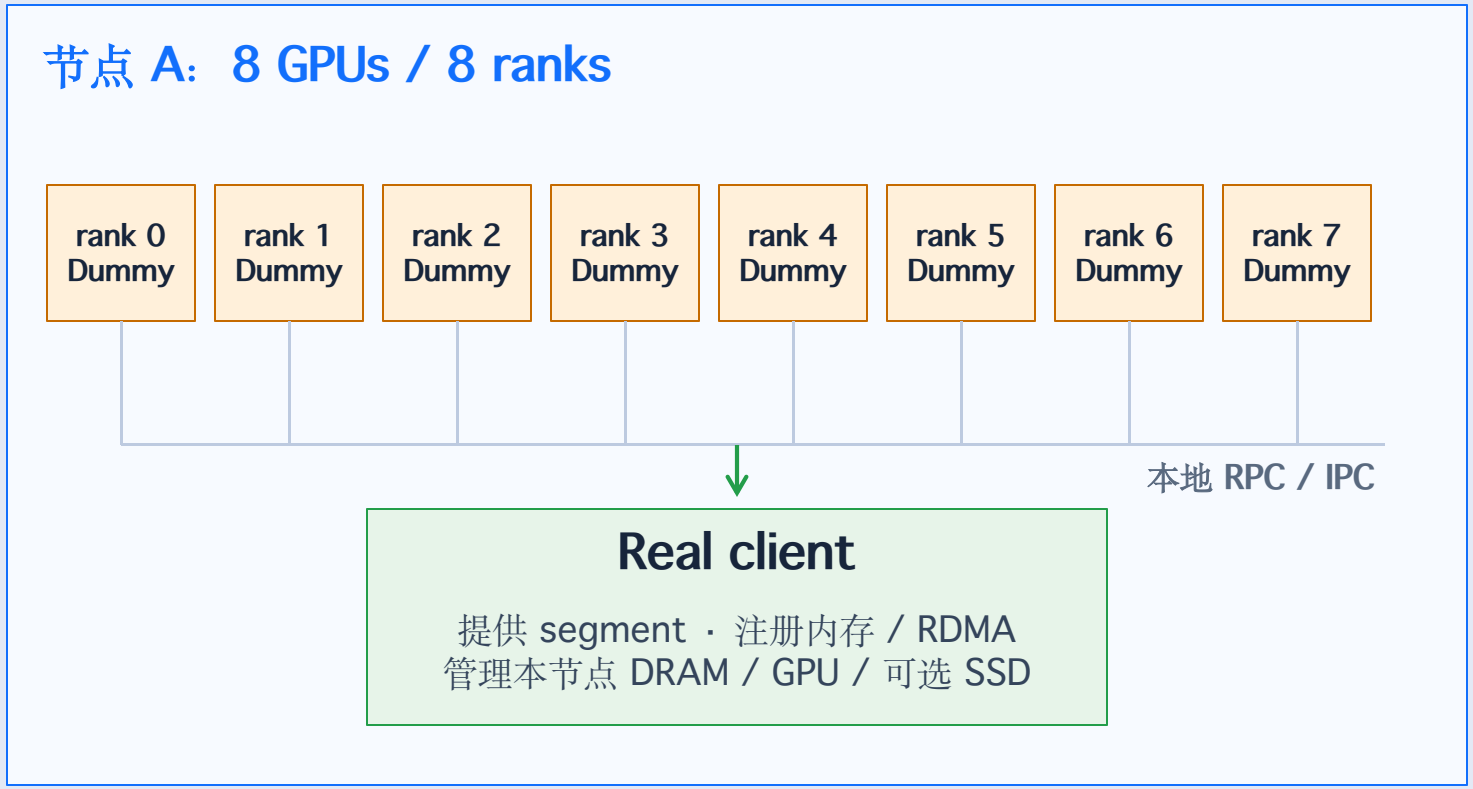
状态 / 选择	需要成立的条件	不应混淆
Source buffer 可复用	所有使用该 buffer 的传输已完成	发出异步写 ≠ 已可覆盖
普通非 in-place 模式	暂停 / flush cache / 后处理 按完成条件使新版本可见	写完成 ≠ 模型已可执行
in_place 特殊语义	保留进行中请求与相应 KV 按其接口约定处理	不承诺自动重算旧 KV
多级 / 远端 cache	检查版本兼容与失效规则	TE 不自动管理 cache 一致性

图示为逻辑成功条件，不强制各接口 RPC 顺序相同；多版本热切换还需要额外容量、引用和退出协议。

QCon 全球软件开发大会
InfoQ 技术大会

Dummy / Real client：线上部署方案

每个 rank 一个调用入口，每个节点一个资源 owner



TE / RDMA

节点 B
Real client
实际 segment owner

Dummy：接口调用，不提供 segment

Real：提供 segment，承担 I/O

Master 维护 metadata / location，不搬运 Payload

同节点可结合 CUDA IPC / pinned memory；跨节点由 Real client 之间使用 TE / RDMA。

QCon 全球软件开发大会
InfoQ 技术大会

04

未来展望

从 RL 数据面走向 Agent、世界模型与推理启动

QCon 全球软件开发大会
InfoQ 技术大会

未来展望：RL 数据流与推理基础设施

近期 RFC：结构化读取、共享容量、设备生命周期和传输安全

事项	具体工作	验收重点
Agentic RL / 世界模型	增量发布工具结果、视频 clip、latent，按字段 / range 消费	端到端等待、数据放大、episode 生命周期
Rollout / Update 分离	版本化 session、共享 Store / TE 数据面、显式关联 policy version	staleness、样本等待、版本正确性
权重管理与快速启动	完整 revision、manifest、lease、迁移与启动加载	model-ready latency、HA 恢复、生命周期正确性
结构化对象批量读取	get_into_ranges 流水 gather，有序 ReadPlan 与计算重叠	有效吞吐、group-ready latency、CPU 开销
共享容量后端	KVCacheStore / Object Storage 成为共享 replica tier	额外 hop、冷启动时间、miss / 回收语义
存储设备故障隔离	NoF device health、cordon / uncordon、graceful drain	新写入隔离、迁移完成、恢复时间
TENT 插件化与安全回收	运行时装载 transport；失败传输提供物理终止凭证	插件准入、P99 / CPU、buffer 安全复用

近期 RFC 主线：结构化读取、共享容量、设备生命周期、传输安全

开放 RFC 以合入版本和真实环境验收为准。

QCon 全球软件开发大会
InfoQ 技术大会

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

Q&A

